

Machine Learning: The No-Nonsense (But Fun) Guide — Day 3

Hyperparameter Tuning: Grid Search CV vs Randomized Search CV

1. What Is a Hyperparameter?

Before we dive in, let's clear the confusion.

Type	What It Is	Example
Parameter	Learned from data during training	Weights (w), Bias (b)
Hyperparameter	Set BEFORE training	Learning rate, Number of trees, Regularization strength

Parameters = Model figures them out.

Hyperparameters = YOU figure them out.

Pick the wrong hyperparameters? Your model is garbage.

Pick the right ones? Magic.

2. Why Do We Need Hyperparameter Tuning?

Imagine baking a cake.

Hyperparameter	Bad Choice	Good Choice
Oven Temperature	50°C (raw) or 500°C (burnt)	180°C (perfect)
Baking Time	5 minutes or 5 hours	30 minutes

Same ingredients, different results.

Your model is the cake.

Hyperparameters are the oven settings.

Example

Model	Hyperparameter	Too Low	Just Right	Too High
Random Forest	n_estimators	Underfitting	Good	Slow, diminishing returns
Logistic Regression	C (regularization)	Overfitting	Good	Underfitting
Neural Network	learning_rate	Never learns	Good	Diverges, unstable

3. The Brute Force Approach: Grid Search CV

Idea

Try EVERY possible combination.

Like a chess computer — exhaustive, thorough, slow.

Hyperparameters

C: [0.1, 1, 10]

penalty: ['l1', 'l2']

Grid Search Tries

(0.1, 'l1')

(0.1, 'l2')

(1, 'l1')

(1, 'l2')

(10, 'l1')

(10, 'l2')

Total Combinations = $3 \times 2 = 6$

Visual Representation

C=0.1 C=1 C=10

GRID SEARCH CV – STEP BY STEP (GRID COMBINATIONS)

Example Hyperparameters

C (Regularization Strength) = [0.01, 0.1, 1, 10]

penalty = ['l1', 'l2']

solver = ['liblinear']

1 Define the hyperparameter grid

C: [0.01, 0.1, 1, 10]
penalty: ['l1', 'l2']
solver: ['liblinear'] (only one option)

2 List all possible values for each hyperparameter

C	penalty	solver
0.01	l1 l2	liblinear
0.1		
1		
10		

3 Compute total combinations

Total combinations =
 $4 \text{ (C values)} \times 2 \text{ (penalty values)} \times 1 \text{ (solver value)}$
= 8 combinations

4 Create the grid of all combinations

penalty ↓	C values →	0.01	0.1	1	10
l1		Try 1	Try 3	Try 5	Try 7
l2		Try 2	Try 4	Try 6	Try 8

5 Enumerate all combinations (what Grid Search will try)

Try #	C	penalty	solver
1	0.01	l1	liblinear
2	0.01	l2	liblinear
3	0.1	l1	liblinear
4	0.1	l2	liblinear
5	1	l1	liblinear
6	1	l2	liblinear
7	10	l1	liblinear
8	10	l2	liblinear

6 Grid Search will evaluate each combination using Cross-Validation



7 Select the combination with the highest average CV score

 **Best Combination**
(C = ?, penalty = ?, solver = liblinear)

Pros

- Guaranteed to find the best combination in your grid
- Simple and reproducible
- Easy to understand

Cons

- Computationally expensive
- Search space grows exponentially
- Wastes time on poor parameter regions

4. The Smart Approach: Randomized Search CV

Idea

Try random combinations instead of all combinations.

Like throwing darts at a dartboard.

Example

Random Picks:

(0.1, 'l2')

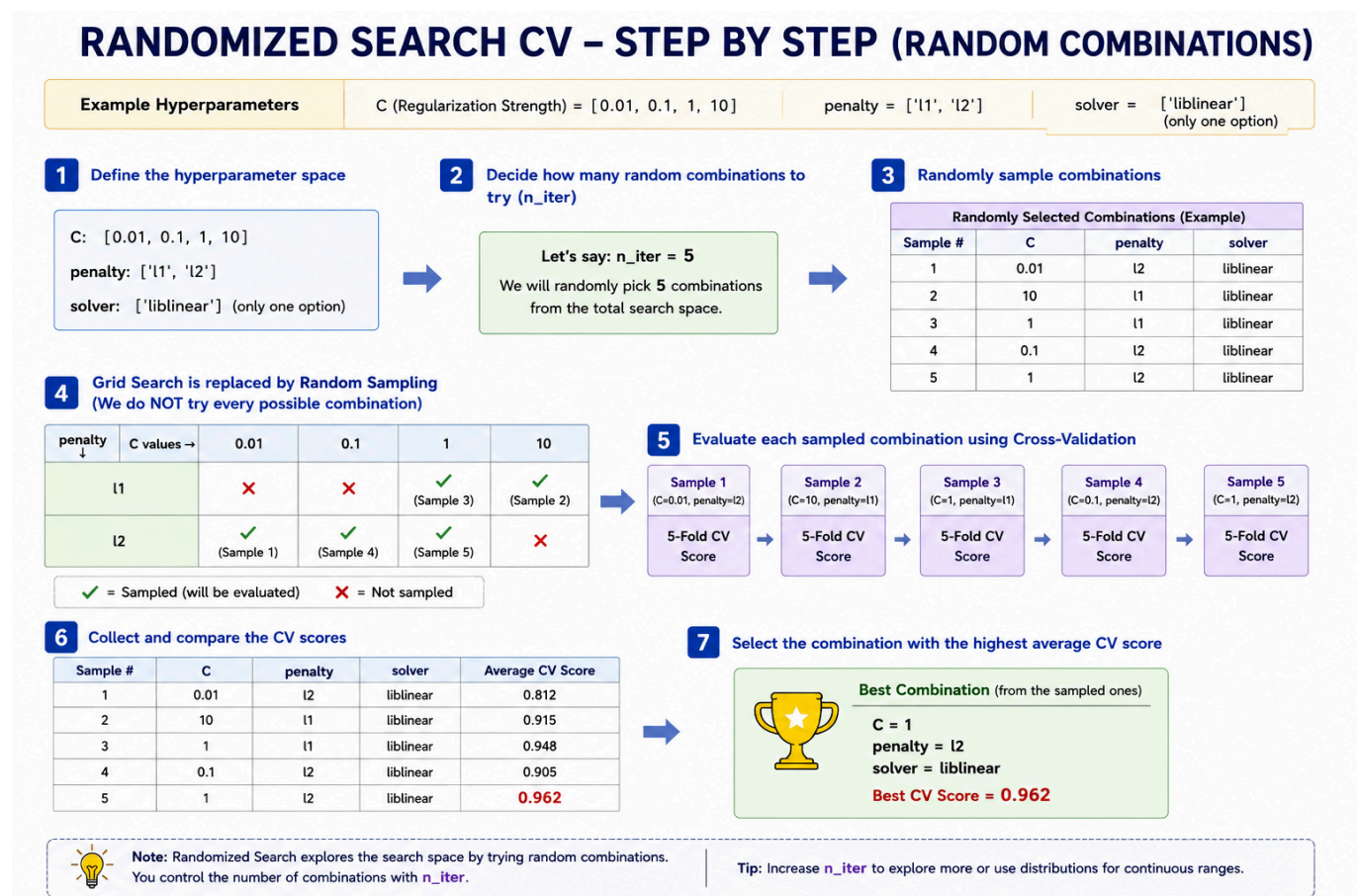
(10, 'l1')

(1, 'l1')

Total combinations tried = 3

Visual Representation

C=0.1 C=1 C=10



Random Picks Only

Pros

- Much faster
- User controls search budget
- Supports continuous distributions
- Scales well

Cons

- May miss the absolute best combination
 - Results vary between runs
-

5. Grid Search CV vs Randomized Search CV

Feature	Grid Search CV	Randomized Search CV
Strategy	Exhaustive Search	Random Sampling
Speed	Slow	Fast
Best For	Small Search Spaces	Large Search Spaces
Guarantee	Finds Best in Grid	No Guarantee
Computation	High	Controlled
Scalability	Poor	Good
Use When	Few Parameters	Many Parameters

6. The "CV" Part: Cross Validation

Both methods use Cross Validation (CV).

5-Fold Cross Validation

Dataset: [■■■■■■■■■■]

Fold 1:

[Test][Train][Train][Train][Train]

Score = 0.82

Fold 2:

[Train][Test][Train][Train][Train]

Score = 0.85

Fold 3:

[Train][Train][Test][Train][Train]

Score = 0.80

Fold 4:

[Train][Train][Train][Test][Train]

Score = 0.83

Fold 5:

[Train][Train][Train][Train][Test]

Score = 0.84

Final Score

Average Score = 0.828

Why CV?

- More reliable than a single train-test split
 - Reduces evaluation variance
 - Helps detect overfitting
-

7. Python Implementation: Grid Search CV

```
from sklearn.model_selection import GridSearchCV
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.datasets import load_iris
```

```
from sklearn.model_selection import train_test_split
```

```
data = load_iris()
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    data.data,
```

```
    data.target,
```

```
    test_size=0.2,
```

```
    random_state=42
```

```
)
```

```
model = LogisticRegression(max_iter=1000)
```

```
param_grid = {  
    'C': [0.001, 0.01, 0.1, 1, 10, 100],  
    'penalty': ['l1', 'l2'],  
    'solver': ['liblinear']  
}
```

```
grid_search = GridSearchCV(  
    estimator=model,  
    param_grid=param_grid,  
    cv=5,  
    scoring='accuracy',  
    n_jobs=-1,  
    verbose=1  
)
```

```
grid_search.fit(X_train, y_train)
```

```
print("Best Parameters:", grid_search.best_params_)
```

```
print("Best CV Score:", grid_search.best_score_)
```

```
print("Test Score:", grid_search.score(X_test, y_test))
```

```
best_model = grid_search.best_estimator_
```

Sample Output

Fitting 5 folds for each of 12 candidates,
totalling 60 fits

Best Parameters:

```
{'C': 1, 'penalty': 'l2', 'solver': 'liblinear'}
```

Best CV Score: 0.9750

Test Score: 0.9667

8. Python Implementation: Randomized Search CV

```
from sklearn.model_selection import RandomizedSearchCV
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
from scipy.stats import randint
```

```
model = RandomForestClassifier(random_state=42)
```

```
param_distributions = {
```

```
    'n_estimators': randint(50, 500),
```

```
    'max_depth': [3, 5, 7, 10, None],
```

```
    'min_samples_split': randint(2, 20),
```

```
    'min_samples_leaf': randint(1, 10),
```

```
'max_features': ['sqrt', 'log2', None]
}

random_search = RandomizedSearchCV(
    estimator=model,
    param_distributions=param_distributions,
    n_iter=50,
    cv=5,
    scoring='accuracy',
    n_jobs=-1,
    random_state=42,
    verbose=1
)
```

```
random_search.fit(X_train, y_train)
```

```
print("Best Parameters:", random_search.best_params_)
```

```
print("Best CV Score:", random_search.best_score_)
```

```
print("Test Score:", random_search.score(X_test, y_test))
```

Sample Output

Fitting 5 folds for each of 50 candidates,

totalling 250 fits

Best Parameters:

```
{'max_depth': 5, 'max_features': 'sqrt', ...}
```

Best CV Score: 0.9667

Test Score: 0.9667

9. When To Use What?

Situation	Recommended Method
Few Hyperparameters (< 4)	Grid Search CV
Many Hyperparameters (> 4)	Randomized Search CV
Limited Compute	Randomized Search CV
Need Guaranteed Best	Grid Search CV
Continuous Search Space	Randomized Search CV
Final Fine-Tuning	Grid Search CV

Pro Tip -

Start with a Randomized Search CV to locate the best region.

Then apply Grid Search CV around that region for fine tuning.

11. Common Hyperparameters to Tune

Model	Important Hyperparameters
Logistic Regression	C, penalty, solver
Random Forest	n_estimators, max_depth, min_samples_split
SVM	C, kernel, gamma
Neural Network	learning_rate, batch_size, epochs, hidden_layers
Gradient Boosting	n_estimators, learning_rate, max_depth

12. Interview Questions

1. What is the difference between a parameter and a hyperparameter?
2. Why do we need hyperparameter tuning?
3. What is the main drawback of Grid Search CV?
4. When would you prefer Randomized Search CV over Grid Search CV?
5. What does the cv parameter do?
6. What is the purpose of n_iter in RandomizedSearchCV?
7. How do you handle a large hyperparameter search space?
8. What is the difference between param_grid and param_distributions?
9. Can you use continuous distributions with Grid Search CV?
10. How do you avoid overfitting during hyperparameter tuning?

Happy Learning !!!